Week 3 Module:

Date: 29/06/2026

Question 1 (Easy):

A university's Training and Placement Cell have successfully completed its annual placement drive. The placement department has collected the salary packages (in LPA) offered to students by different companies. However, the records were entered in the order they were received, resulting in an unsorted list of package values.

Before publishing the placement statistics report, the department wants all package values arranged in ascending order. Since the university plans to handle placement data for thousands of students in the coming years, they want a sorting technique that is efficient for large datasets.

Your task is to implement the **Merge Sort algorithm** to sort the package values in increasing order. Merge Sort follows a divide-and-conquer approach by dividing the dataset into smaller subarrays, sorting them recursively, and then merging them back together.

The final sorted list will help the university identify minimum, median, and maximum packages while generating analytical reports.

Input

- First line contains an integer **N**, representing the number of placed students.
- Second line contains **N space-separated integers**, representing package values (multiplied by 100 to avoid decimals).

Output

Print the package values in ascending order.

Constraints

- $1 \leq N \leq 100000$
- $100 \leq \text{Package} \leq 10000$

Example

Input:

6
450 1200 800 650 300 1500

Output:

300 450 650 800 1200 1500

Question 2 (Medium):

A large e-commerce company tracks the time taken (in minutes) to process customer orders before shipment. Due to varying warehouse workloads, processing times differ significantly from one order to another.

The analytics team wants to study operational efficiency by sorting all processing times and generating statistical information. Since millions of orders are processed every month, the company requires an efficient sorting technique with predictable performance.

Your task is to use **Merge Sort** to arrange all processing times in ascending order.
After sorting, perform the following operations:

1. Display the sorted processing times.
2. Find the **median processing time**.
3. Count the number of orders whose processing time is greater than the median.
4. Display the difference between the fastest and slowest processing time.

The solution should efficiently handle large datasets and demonstrate a clear understanding of Merge Sort's divide-and-conquer strategy.

Input

- First line contains integer **N**.
- Second line contains **N space-separated processing times**.

Output

- Sorted processing times.
- Median processing time.
- Count of orders greater than median.
- Difference between maximum and minimum processing time.

Constraints

- $1 \leq N \leq 100000$
- $1 \leq Time \leq 100000$

Example

Input:

7
12 25 18 30 15 22 40

Output:

12 15 18 22 25 30 40
Median: 22
Orders Above Median: 3
Difference: 28

Topic 2: Quick Sort

Question 1 (Easy):

A cloud infrastructure company manages hundreds of servers distributed across different geographical locations. Every hour, each server reports its average response time in milliseconds. The collected data is stored in the order it arrives and is not sorted.

To generate performance reports, the operations team wants the response times arranged from the fastest server to the slowest server. Since the data size can become very large, an efficient in-place sorting technique is required.

Your task is to implement **Quick Sort** to sort the response times in ascending order. Built-in sorting methods are not allowed.

After sorting, the operations team will use the report to identify high-latency servers and optimize system performance.

Input

- First line contains integer **N**.
- Second line contains **N space-separated response times**.

Output

Print the response times in ascending order.

Constraints

- $1 \leq N \leq 100000$
- $1 \leq$ Response Time ≤ 10000

Example

Input:

6
120 45 78 200 60 95

Output:

45 60 78 95 120 200

Question 2 (Medium):

A financial analytics company receives daily trade-value data from multiple stock exchanges. Analysts use this information to identify high-value trading activity and market trends.

The trade values are received in random order throughout the day. Before analysis can begin, the data must be sorted in descending order.

Your task is to implement **Quick Sort** to arrange the trade values from highest to lowest.

After sorting, perform the following operations:

1. Display the sorted trade values.
2. Find the **Top 5 highest trade values**.
3. Calculate the average of the Top 5 values.
4. Count how many trade values are greater than the overall average of all trade values.

This problem evaluates your understanding of partitioning, recursion, and real-world data analysis using Quick Sort.

Input

- First line contains integer **N**.
- Second line contains **N space-separated trade values**.

Output

- Sorted trade values (descending).
- Top 5 values.
- Average of Top 5 values.
- Count of values greater than overall average.

Constraints

- $5 \leq N \leq 100000$
- $1 \leq$ Trade Value $\leq 10^9$

Example

Input:

8
500 1200 700 2000 900 1500 3000 2500

Output:

3000 2500 2000 1500 1200 900 700 500
Top 5: 3000 2500 2000 1500 1200
Average of Top 5: 2040
Values Above Overall Average: 4

Topic 3: Heap Sort

Question 1 (Easy):

An online gaming platform is hosting a national-level tournament involving thousands of participants. At the end of the tournament, player scores are collected from multiple game servers and stored in random order.

Before announcing the final rankings, the organizers need all scores sorted in ascending order. Since the leaderboard data can be very large, they have decided to use **Heap Sort**, which provides guaranteed O(N log N) performance.

Your task is to implement Heap Sort and arrange all player scores in increasing order.

The sorted output will help tournament officials generate rankings and determine qualification thresholds for future events.

Input

- First line contains integer **N**.
- Second line contains **N space-separated player scores**.

Output

Print the scores in ascending order.

Constraints

- $1 \leq N \leq 100000$
- $0 \leq Score \leq 1000000$

Example

Input:

6
850 1200 950 600 1500 1000

Output:

600 850 950 1000 1200 1500

Question 2 (Medium):

A multi-specialty hospital maintains records of emergency response times for all critical cases handled during a month. Each response time represents the number of minutes taken by the emergency team to reach and begin treatment for a patient.

Hospital administrators want to analyze the efficiency of their emergency services. To do so, all response times must first be sorted using **Heap Sort**.

After sorting the response times in ascending order, generate the following statistics:

1. Display the sorted response times.
2. Find the fastest response time.
3. Find the slowest response time.
4. Calculate the average response time.
5. Count the number of cases handled faster than the average.
6. Calculate the percentage of cases handled faster than the average.

The hospital intends to use this information to improve resource allocation and reduce emergency response delays.

Input

- First line contains integer **N**.
- Second line contains **N space-separated response times**.

Output

- Sorted response times.
- Fastest response time.
- Slowest response time.
- Average response time.
- Number of cases faster than average.
- Percentage of cases faster than average.

Constraints

- $1 \leq N \leq 100000$
- $1 \leq Time \leq 500$

Example

Input:

8
12 7 15 9 20 5 10 8

Output:

5 7 8 9 10 12 15 20
Fastest: 5
Slowest: 20
Average: 10.75
Cases Faster Than Average: 5
Percentage: 62.50%

MODULE 3

Page No.
Date

(1) Algo

1) Read the no. of elements N.
2) Read the array elements.
3) Call mergeSort(arr, 0, N-1);
4) In mergeSort():
   if left < right:
   - find the middle index
   - Recursively sort the left half.
   - Recursively sort the right half.
   - Merge both sorted halves using merge().
5) In merge():
   - Create 2 temp array for left & right halves.
   - Copy the elements into temp arrays.
   - Compare elements from both arrays & place smaller element into original array.
   - Copy remaining element from left or right array.
6) Print the sorted array.

(2) Algo

1) Read the no. of elements N
2) Read all array elements.
3) Sort the array using merge sort
4) Print the sorted array.
5) Find the median:
   - If N is median = middle element
   - If N is even, median = avg of 2 middle element
6) Traverse the array and count the no. of elements greater than median.
7) Find the diff. b/w largest & smallest element
8) Print: sorted array, median, elements greater than median, difference,

(3) Algo

1) Read the no. of elements N.
2) Read all array elements.
3) Call quickSort(arr, 0, N-1).
4) In quickSort(),
   If low < high:
   - find pivot pos. using partition().
   - Recursively sort left subarray.
   - Recursively sort right subarray.
5) In partition(),
   - select last element as pivot.
   - Initialize i = low - 1.
   - Traverse array from low to high-1.
   - If current element <= pivot:
     - increment i
     - swap(arr[i], arr[j]).
   - After traversal, swap pivot with arr[i+1].
   - Return pivot index.
6) Print sorted array.

(4) Algo

1) Read no. of elements N.
2) Read all trade values & calculate their total sum.
3) Sort array in desc. order using quick sort.
4) In partition():
   - select last element as pivot.
   - initialize i = low - 1.
   - Traverse array from low to high-1.
   - If curr element >= pivot
     - increment i
     - swap(arr[i], arr[j]);

- Place the pivot at correct pos.
5) Print the sorted array.
6) Print first 5 elements or top 5 values.
7) Calculate avg of top 5 values.
8) Calculate overall avg of all node values.
9) Traverse array and count the values greater than overall avg.
10) Print : sorted array, Top 5 values, avg of top 5, no. > overall avg.

(5) Algo
1) Read no. of elements N.
2) Read all array elements.
3) Build a maxheap from array.
4) Repeat until only one element remains in the heap.
   - Swap the root with last element.
   - Reduce heap size by 1.
   - Apply heapify() on root to restore max heap property.
5) After all iteration, array will be sorted in asc. order.
6) Print the sorted array.

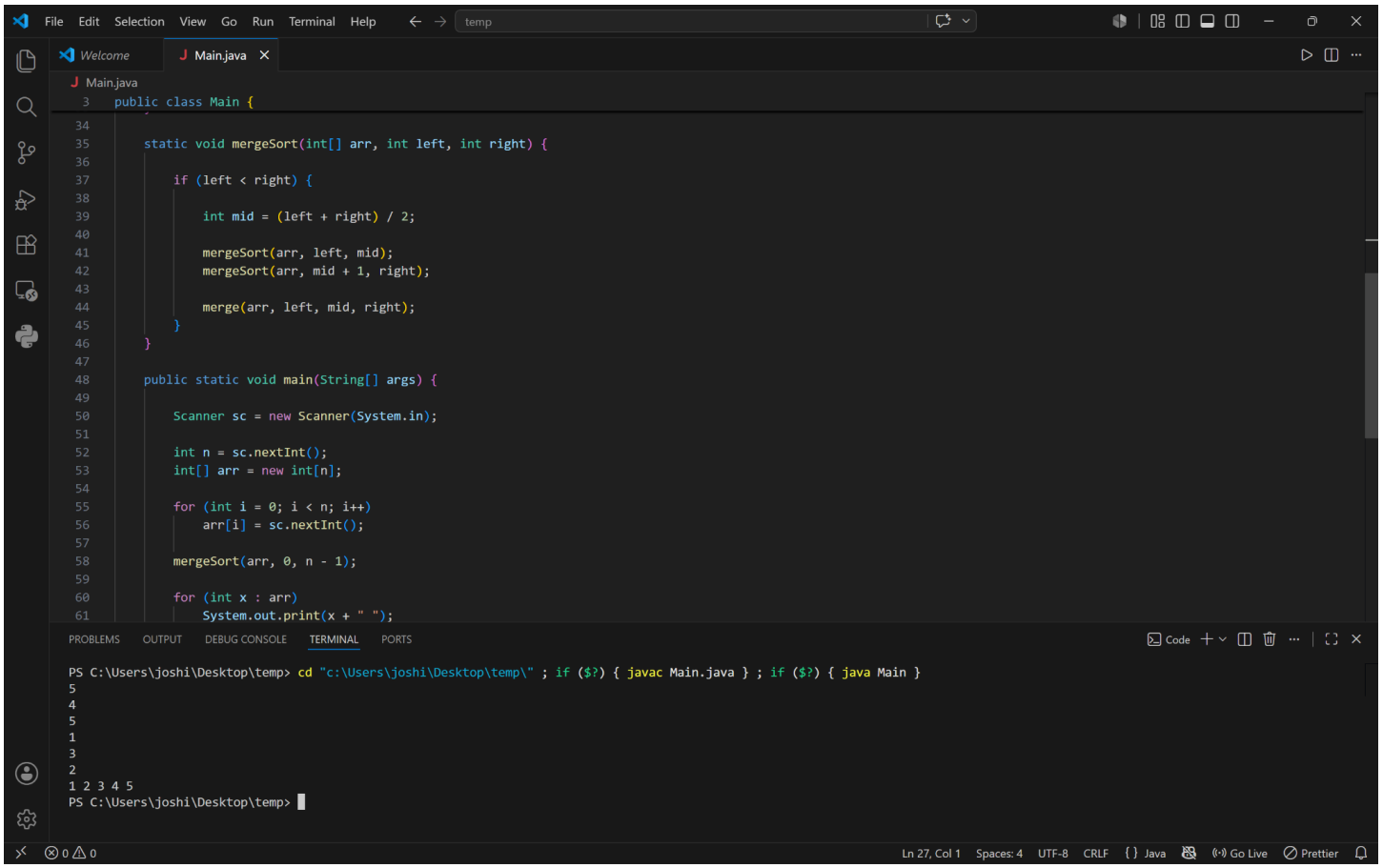
(6) Algo
1) Read the no. of elements N.
2) Read all response times and calculate their total sum.
3) Sort the array using heap sort.
4) Print sorted array.
5) find fastest response time. (first element)
6) find slowest response time (last element).
7) Calculate avg response time
   Avg = Sum / N

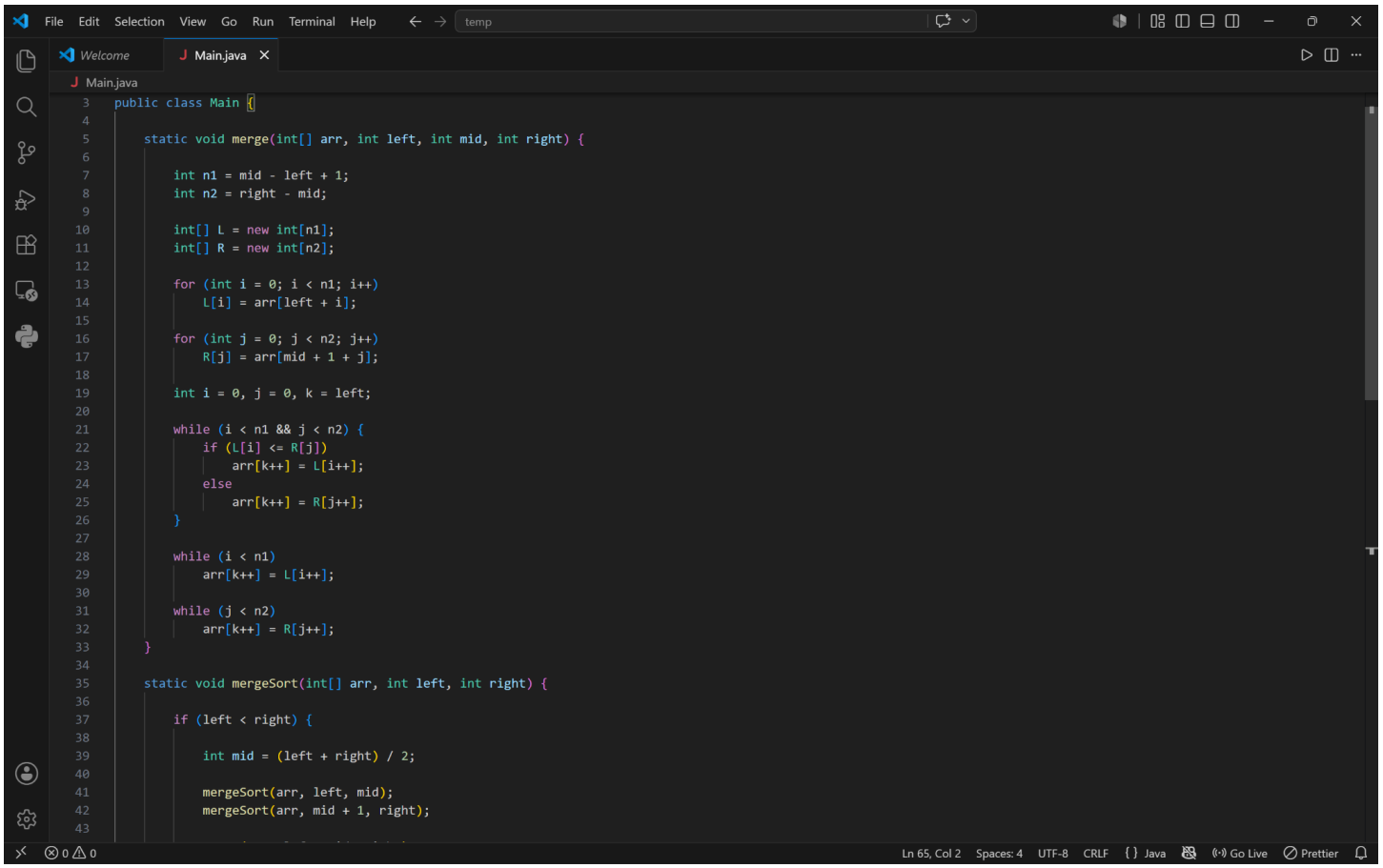
8) Traverse the sorted array and count no. of response times less than avg.

9) Calculate % of cases handled faster than avg using: Percentage = (count × 100) / N.

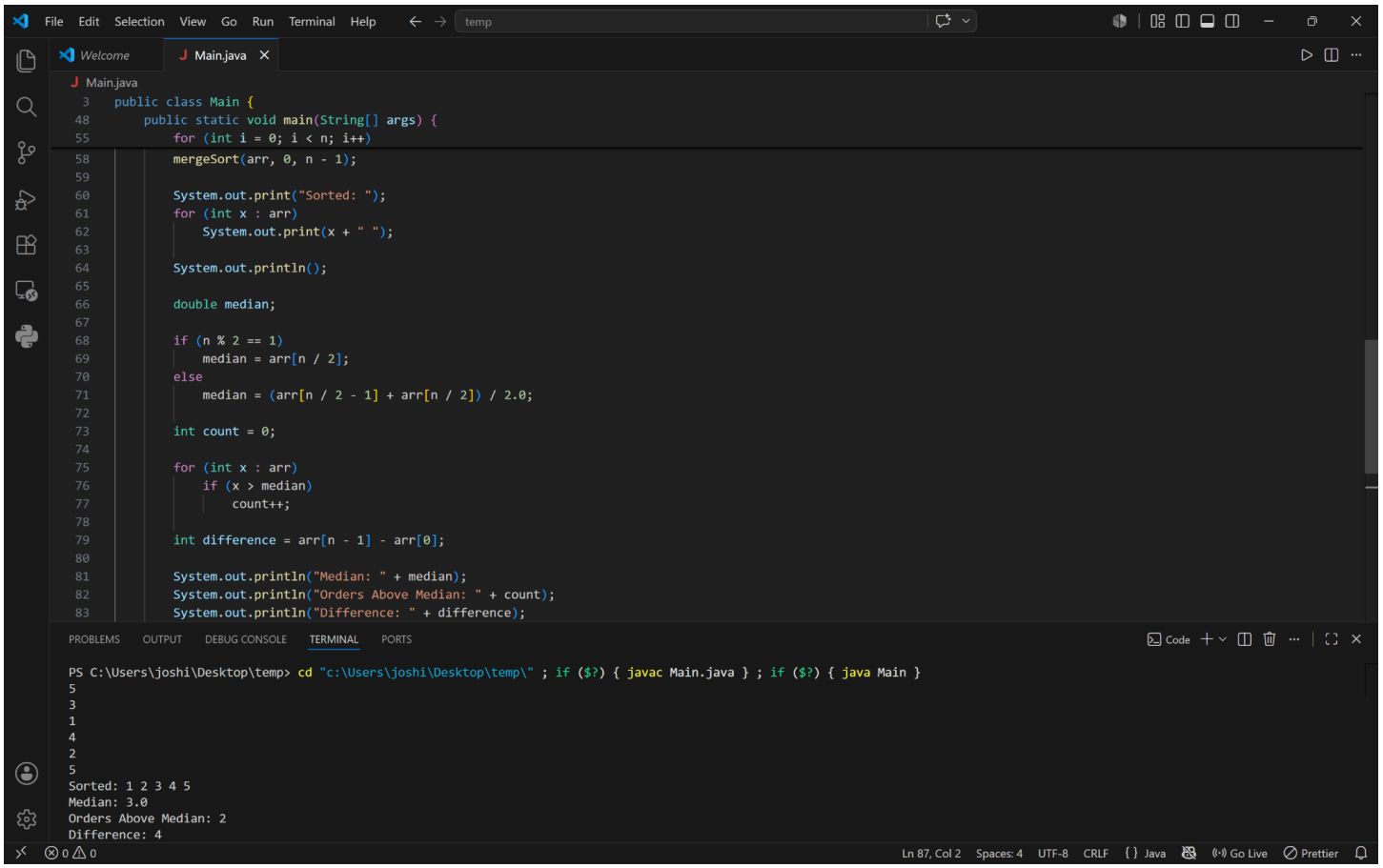
10) Print: sorted response times, fastest one, slowest one, avg response time, cases faster than avg, % of cases faster than avg.

Q1 - Merge Sort (Package Values Sorting)

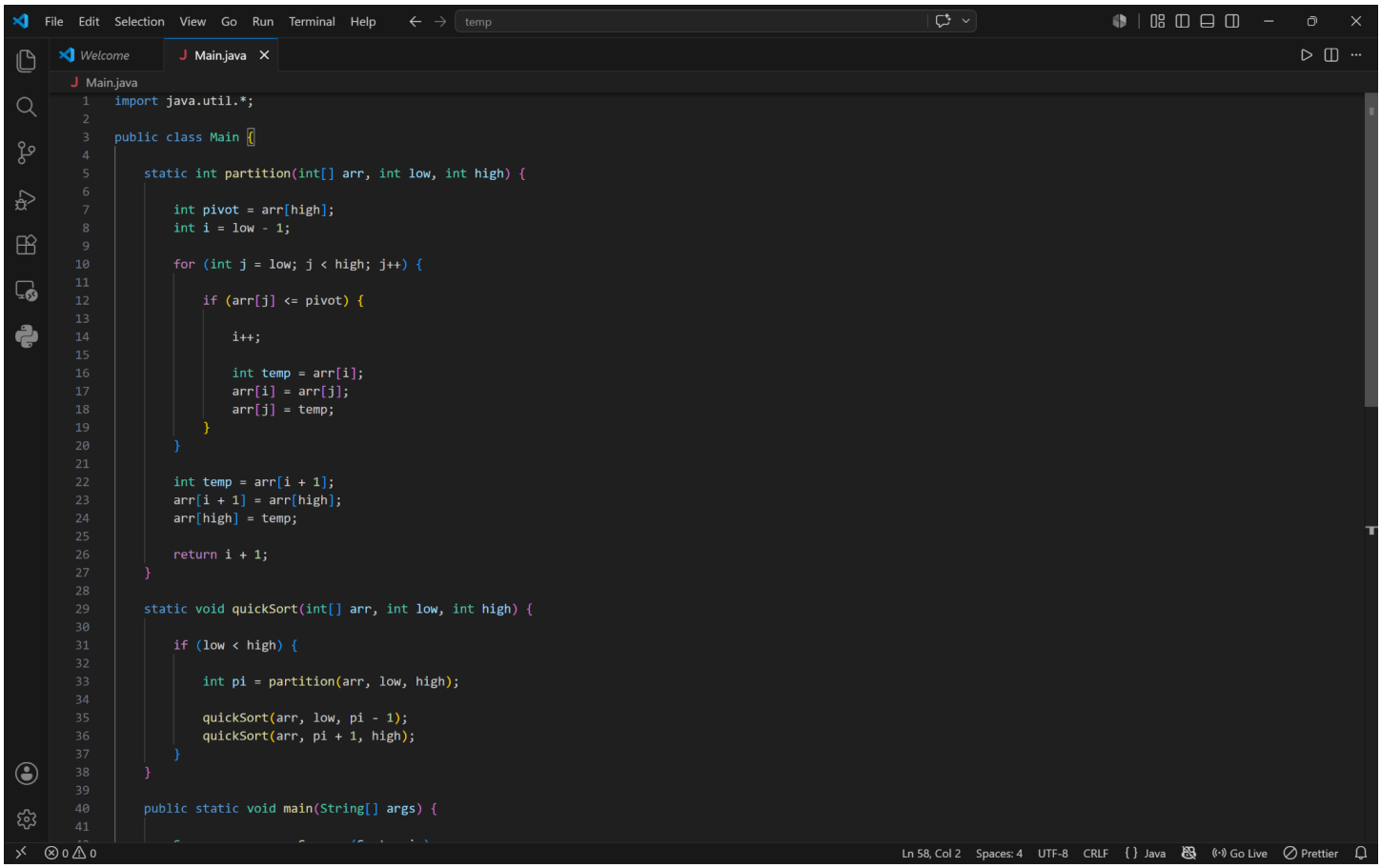


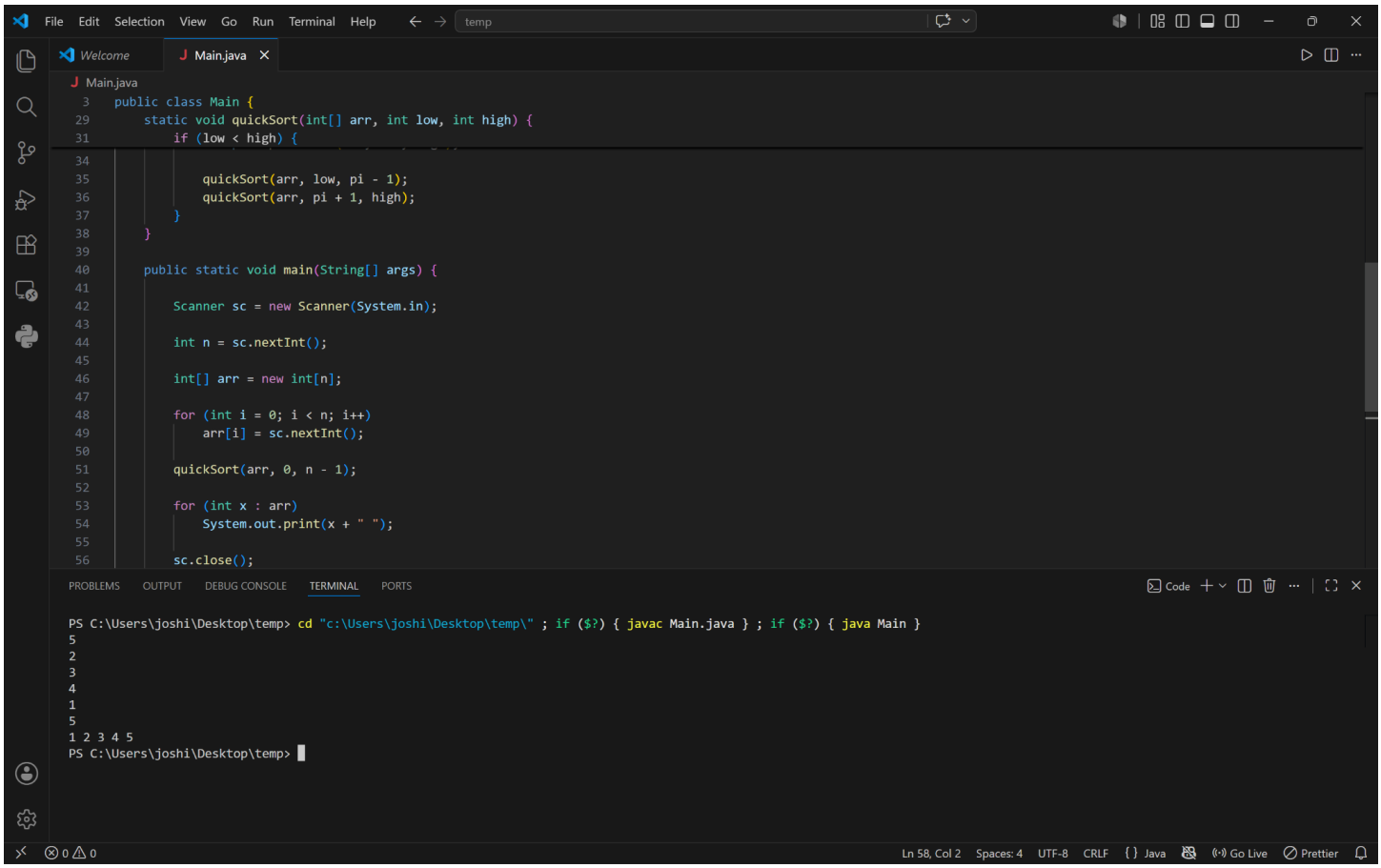

File Edit Selection View Go Run Terminal Help
temp
Welcome
Main.java
Main.java
public class Main {
static void merge(int[] arr, int left, int mid, int right) {
int n1 = mid - left + 1;
int n2 = right - mid;
int[] L = new int[n1];
int[] R = new int[n2];
for (int i = 0; i < n1; i++)
L[i] = arr[left + i];
for (int j = 0; j < n2; j++)
R[j] = arr[mid + 1 + j];
int i = 0, j = 0, k = left;
while (i < n1 && j < n2) {
if (L[i] <= R[j])
arr[k++] = L[i++];
else
arr[k++] = R[j++];
}
while (i < n1)
arr[k++] = L[i++];
while (j < n2)
arr[k++] = R[j++];
}
static void mergeSort(int[] arr, int left, int right) {
if (left < right) {
int mid = (left + right) / 2;
mergeSort(arr, left, mid);
mergeSort(arr, mid + 1, right);
0 0
Ln 65, Col 2
Spaces: 4
UTF-8
CRLF
Java
Go Live
Prettier

Q2 - Merge Sort (Processing Time Analysis)

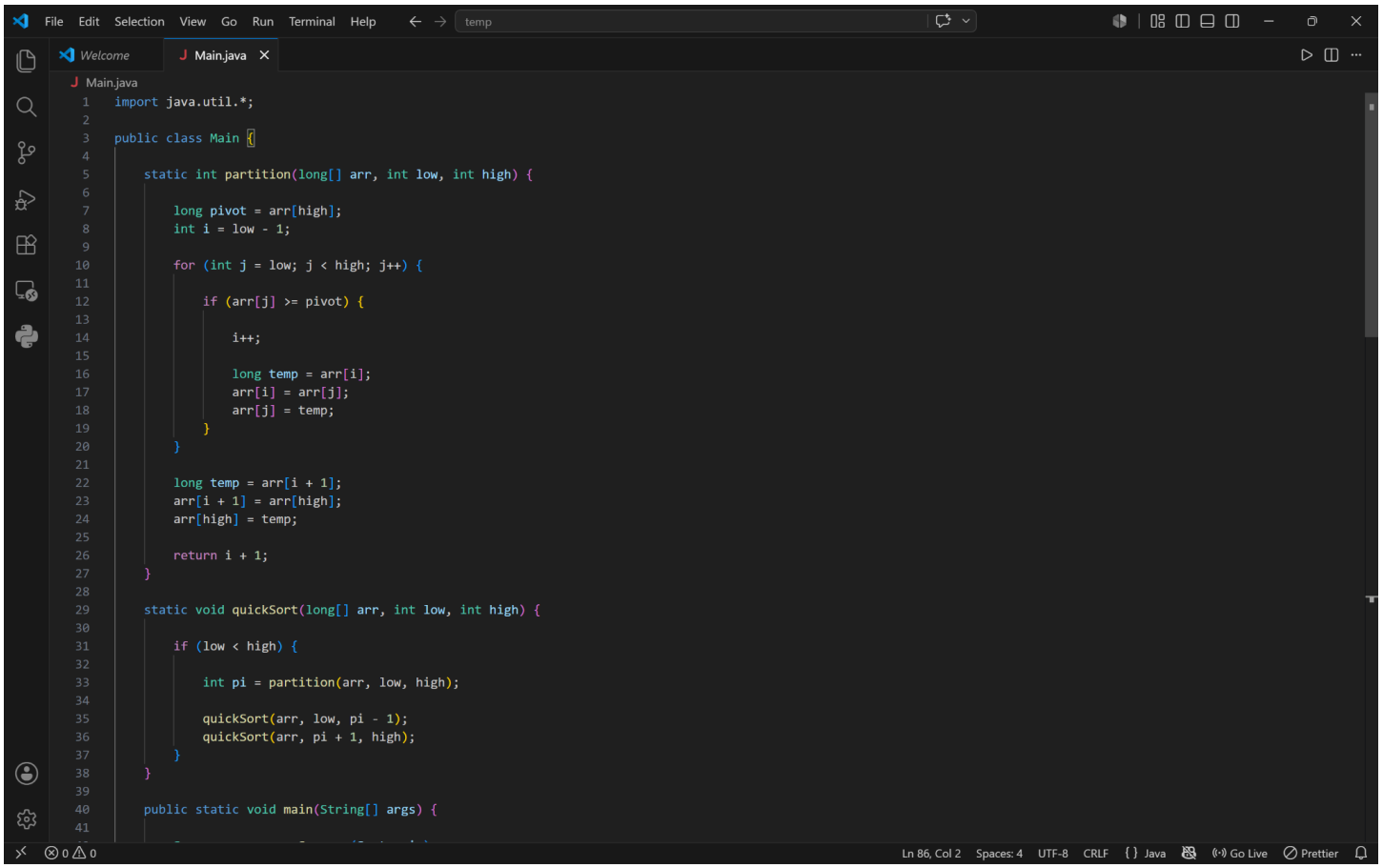


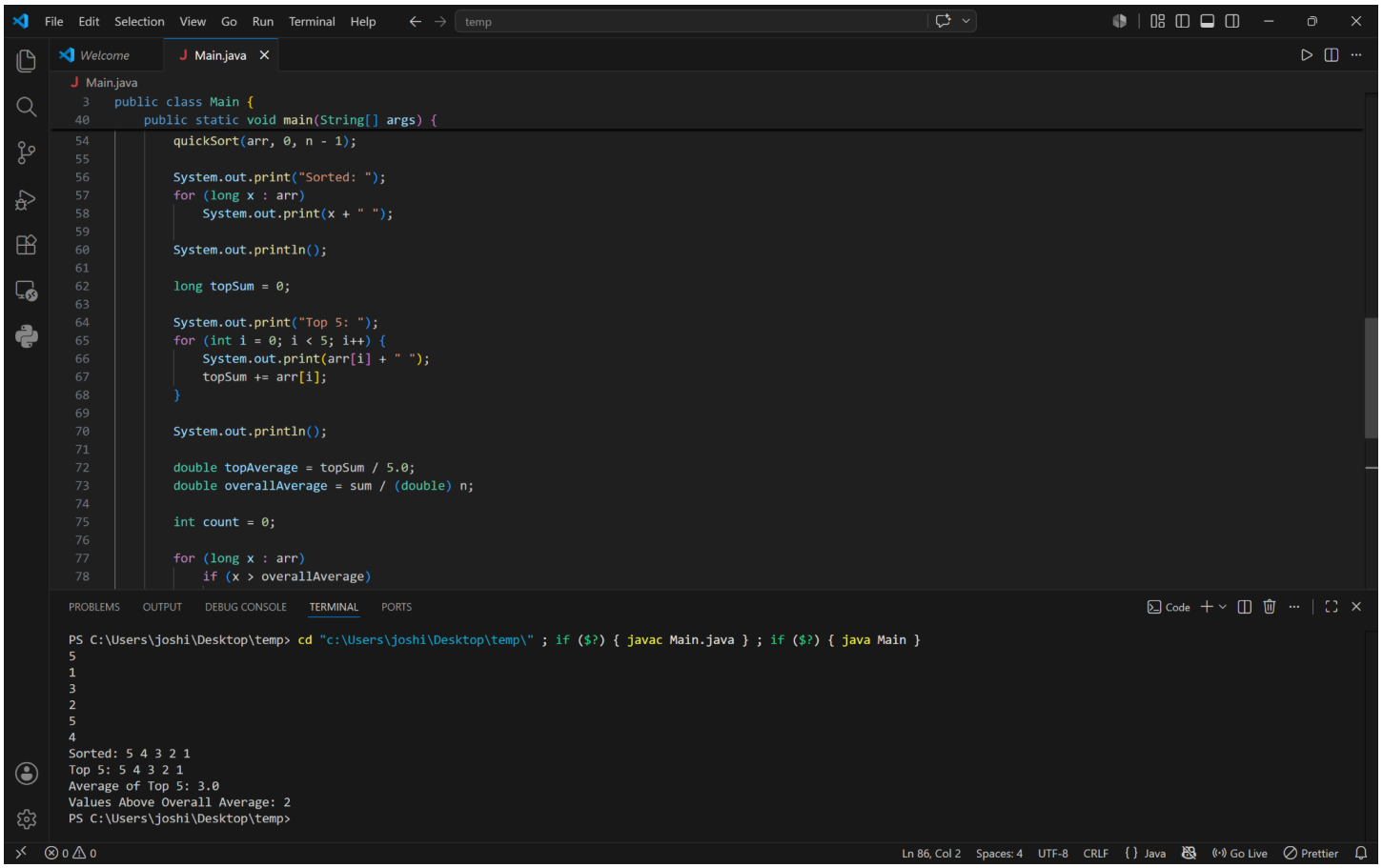
Q3 - Quick Sort (Server Response Time Sorting)



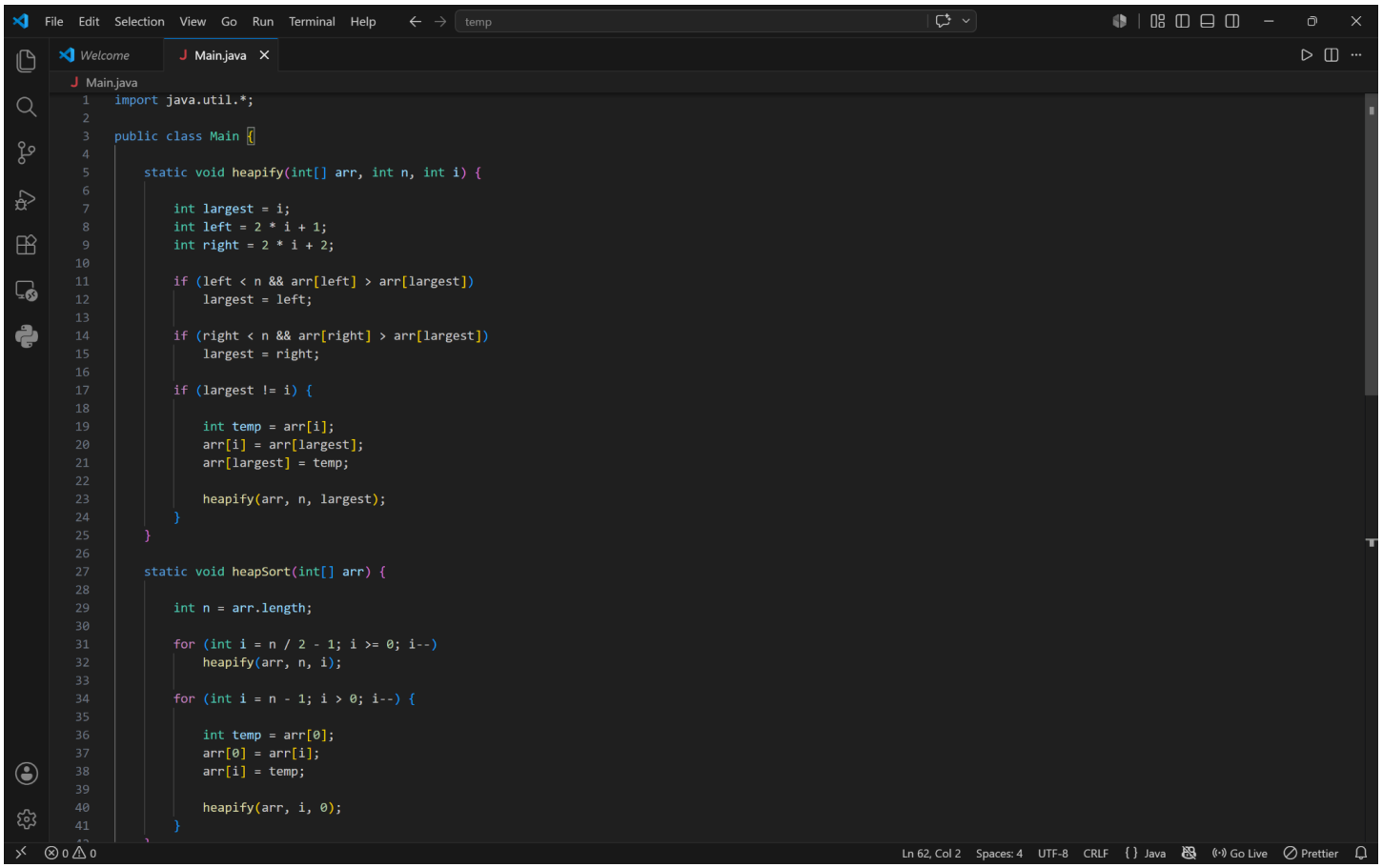

File Edit Selection View Go Run Terminal Help
temp
Welcome
Main.java
Main.java
3 public class Main {
29 static void quickSort(int[] arr, int low, int high) {
31 if (low < high) {
34
35 quickSort(arr, low, pi - 1);
36 quickSort(arr, pi + 1, high);
37 }
38 }
39
40 public static void main(String[] args) {
41
42 Scanner sc = new Scanner(System.in);
43
44 int n = sc.nextInt();
45
46 int[] arr = new int[n];
47
48 for (int i = 0; i < n; i++)
49 arr[i] = sc.nextInt();
50
51 quickSort(arr, 0, n - 1);
52
53 for (int x : arr)
54 System.out.print(x + " ");
55
56 sc.close();
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Code
PS C:\Users\joshi\Desktop\temp> cd "c:\Users\joshi\Desktop\temp\" ; if ($?) { javac Main.java } ; if ($?) { java Main }
5
2
3
4
1
5
1 2 3 4 5
PS C:\Users\joshi\Desktop\temp>
0 0
Ln 58, Col 2 Spaces: 4 UTF-8 CRLF { } Java Go Live Prettier

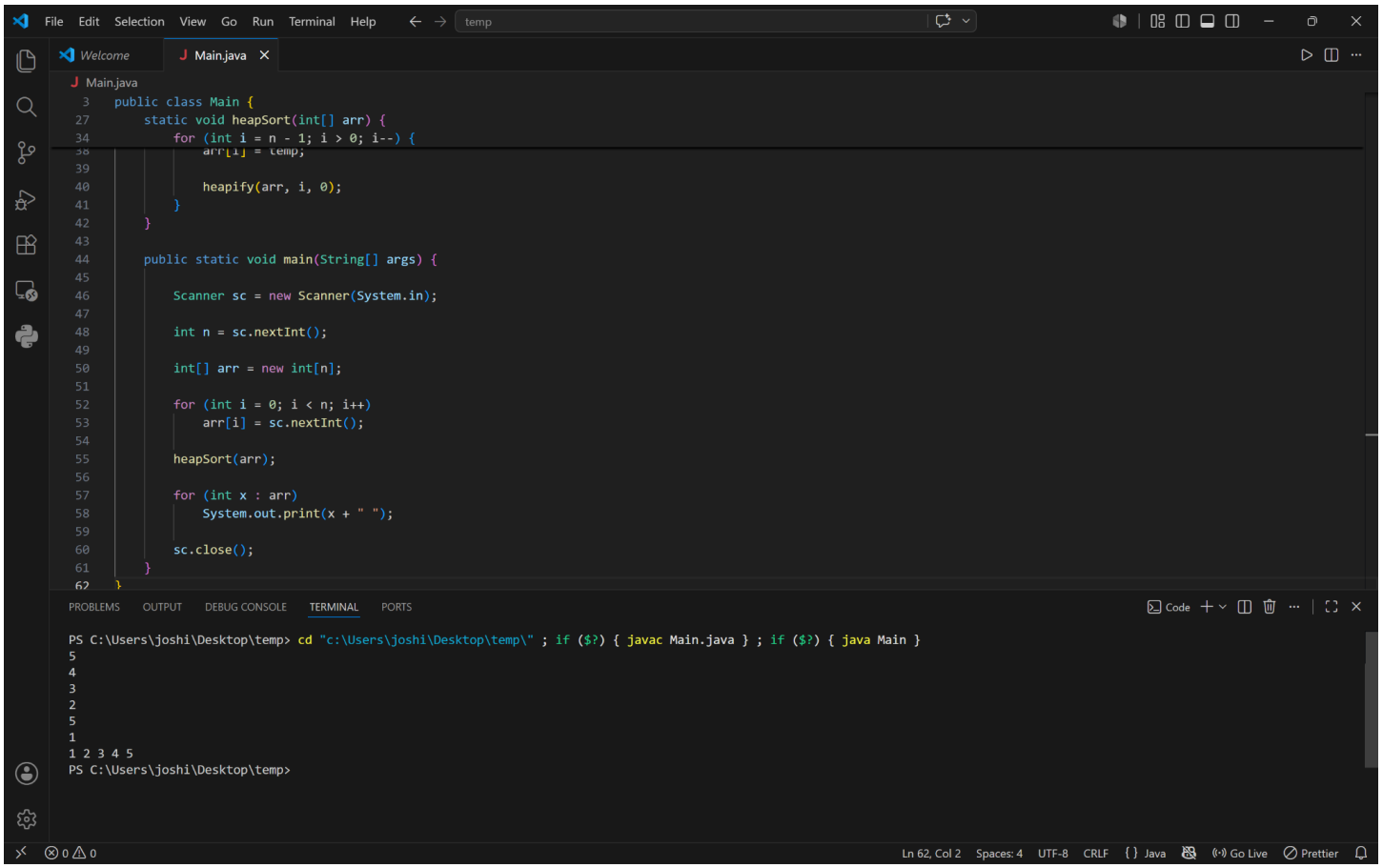
Q4 - Quick Sort (Trade Value Analysis)



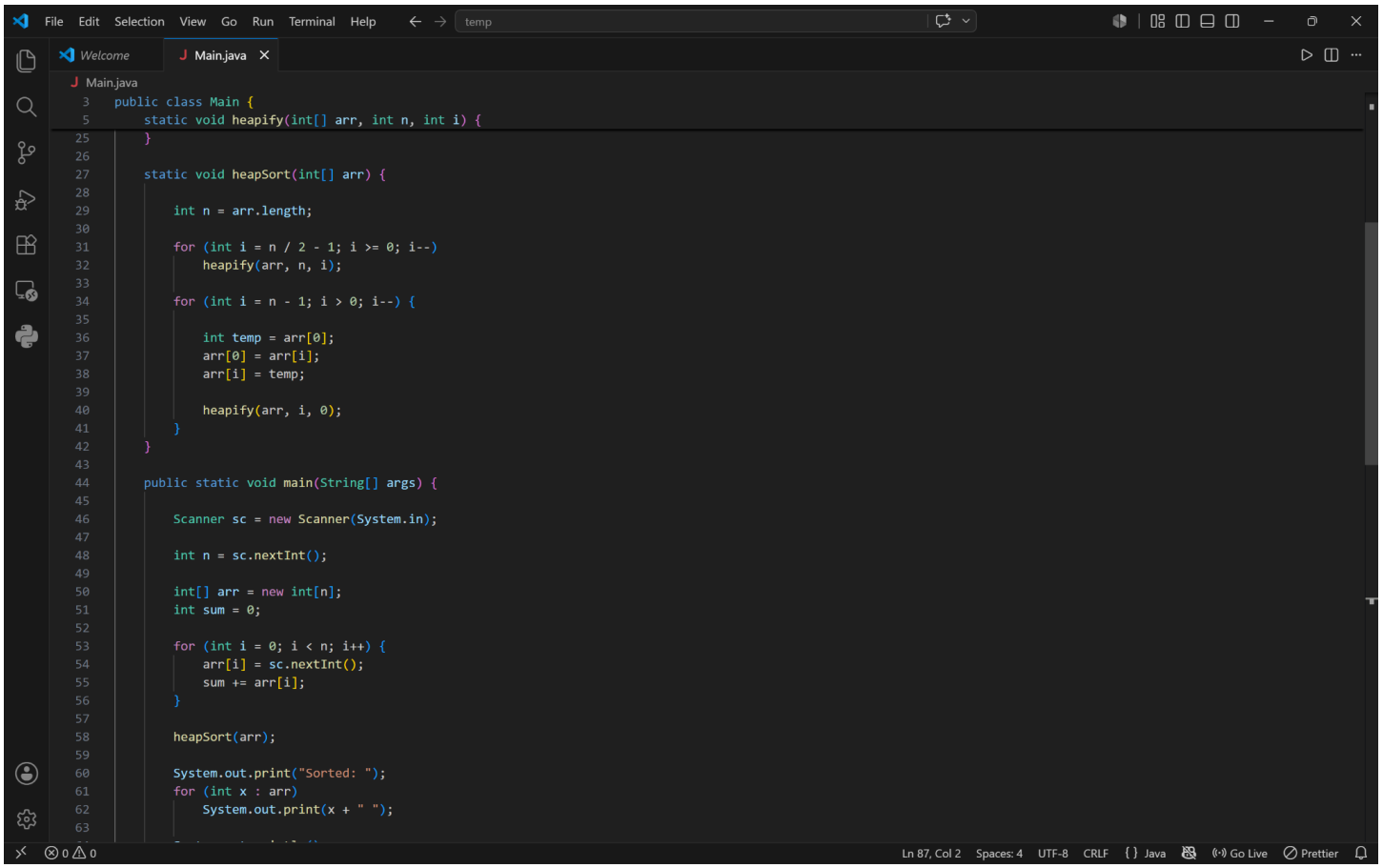

File Edit Selection View Go Run Terminal Help
temp
Welcome
Main.java
Main.java
3 public class Main {
40 public static void main(String[] args) {
54 quickSort(arr, 0, n - 1);
56 System.out.print("Sorted: ");
57 for (long x : arr)
58 System.out.print(x + " ");
60 System.out.println();
62 long topSum = 0;
64 System.out.print("Top 5: ");
65 for (int i = 0; i < 5; i++) {
66 System.out.print(arr[i] + " ");
67 topSum += arr[i];
68 }
70 System.out.println();
72 double topAverage = topSum / 5.0;
73 double overallAverage = sum / (double) n;
75 int count = 0;
77 for (long x : arr)
78 if (x > overallAverage)
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Code
PS C:\Users\joshi\Desktop\temp> cd "c:\Users\joshi\Desktop\temp\" ; if ($?) { javac Main.java } ; if ($?) { java Main }
5
1
3
2
5
4
Sorted: 5 4 3 2 1
Top 5: 5 4 3 2 1
Average of Top 5: 3.0
Values Above Overall Average: 2
PS C:\Users\joshi\Desktop\temp>
0 0
Ln 86, Col 2 Spaces: 4 UTF-8 CRLF { } Java Go Live Prettier

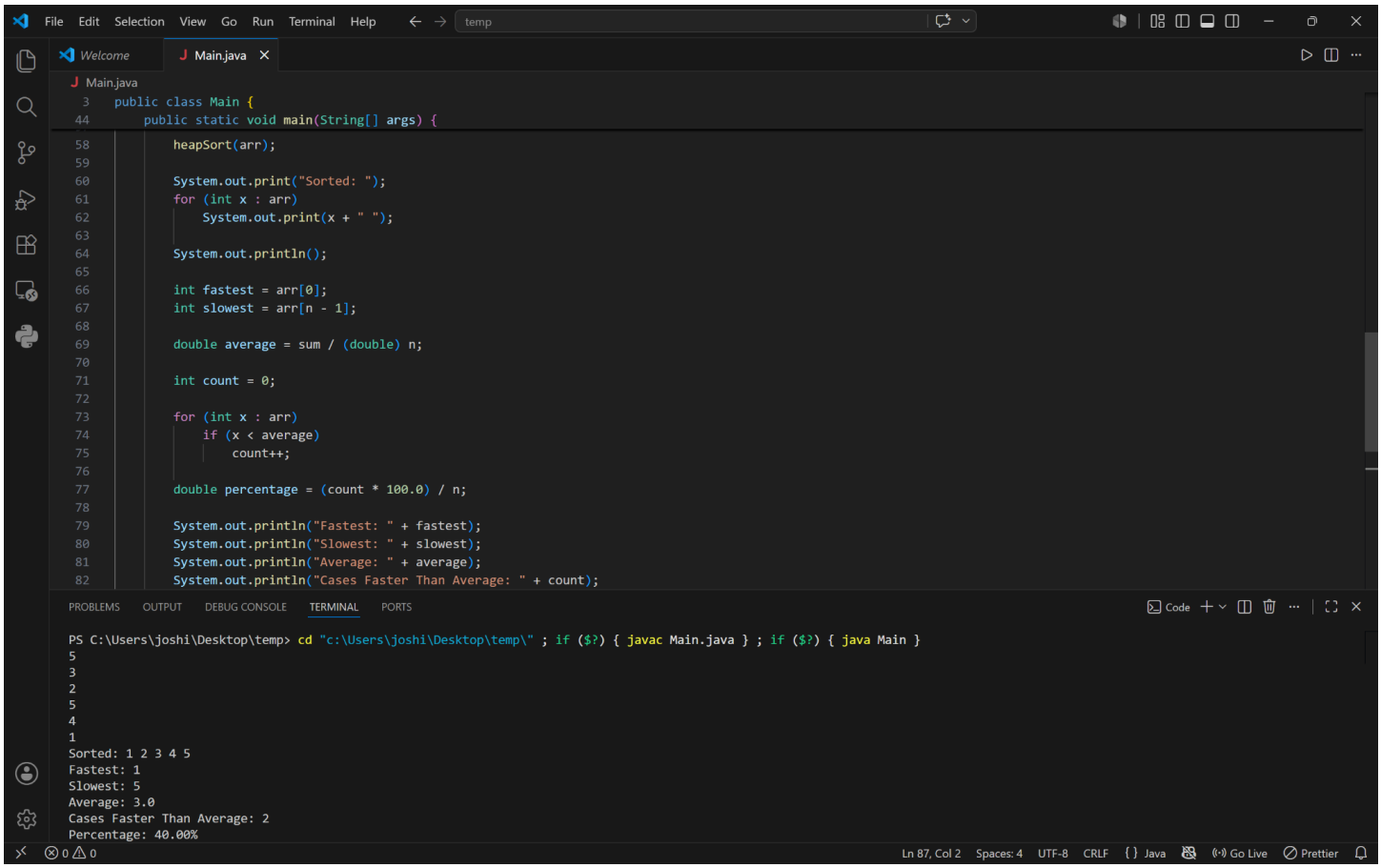
Q5 - Heap Sort (Player Score Sorting)




File Edit Selection View Go Run Terminal Help
temp
Welcome
Main.java
Main.java
public class Main {
static void heapSort(int[] arr) {
for (int i = n - 1; i > 0; i--) {
arr[i] = temp;
heapify(arr, i, 0);
}
}
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
int n = sc.nextInt();
int[] arr = new int[n];
for (int i = 0; i < n; i++)
arr[i] = sc.nextInt();
heapSort(arr);
for (int x : arr)
System.out.print(x + " ");
sc.close();
}
}
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
Code
PS C:\Users\joshi\Desktop\temp> cd "c:\Users\joshi\Desktop\temp\" ; if ($?) { javac Main.java } ; if ($?) { java Main }
5
4
3
2
5
1
1 2 3 4 5
PS C:\Users\joshi\Desktop\temp>
Ln 62, Col 2
Spaces: 4
UTF-8
CRLF
Java
Go Live
Prettier

Q6 - Heap Sort (Hospital Response Time Analysis)




File Edit Selection View Go Run Terminal Help
temp
Welcome
Main.java
Main.java
3 public class Main {
44 public static void main(String[] args) {
58 heapSort(arr);
59
60 System.out.print("Sorted: ");
61 for (int x : arr)
62 System.out.print(x + " ");
63
64 System.out.println();
65
66 int fastest = arr[0];
67 int slowest = arr[n - 1];
68
69 double average = sum / (double) n;
70
71 int count = 0;
72
73 for (int x : arr)
74 if (x < average)
75 count++;
76
77 double percentage = (count * 100.0) / n;
78
79 System.out.println("Fastest: " + fastest);
80 System.out.println("Slowest: " + slowest);
81 System.out.println("Average: " + average);
82 System.out.println("Cases Faster Than Average: " + count);
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Code
PS C:\Users\joshi\Desktop\temp> cd "c:\Users\joshi\Desktop\temp\" ; if ($?) { javac Main.java } ; if ($?) { java Main }
5
3
2
5
4
1
Sorted: 1 2 3 4 5
Fastest: 1
Slowest: 5
Average: 3.0
Cases Faster Than Average: 2
Percentage: 40.00%
0 0
Ln 87, Col 2 Spaces: 4 UTF-8 CRLF { } Java Go Live Prettier